

General Notes

- You will submit a minimum of three files, the core files must conform to the following naming conventions (including capitalization and underscores). 123456789 is a placeholder, please replace these nine digits with your nine-digit Bruin ID. The files you must submit are:
 1. *123456789_stats102a_hw3.R*: An R script file containing all of the functions you wrote for the homework. The first line of your .Rmd file, **after loading libraries, should be sourcing this script file**.
 2. *123456789_stats102a_hw3.Rmd*: Your markdown file which generates the output file of your submission.
 3. *123456789_stats102a_hw3.html/pdf*: Your output file, either a PDF or an HTML file depending on the output you choose to generate.
 4. Please place all of your files into a single folder named 123456789_stats102a_hw3 and compress the folder into 123456789_stats102a_hw3.zip.

If you fail to submit any of the required core files you will receive **ZERO** points for the assignment. If you submit any files which do not conform to the specified naming convention, you will receive **half credit** for the assignment.

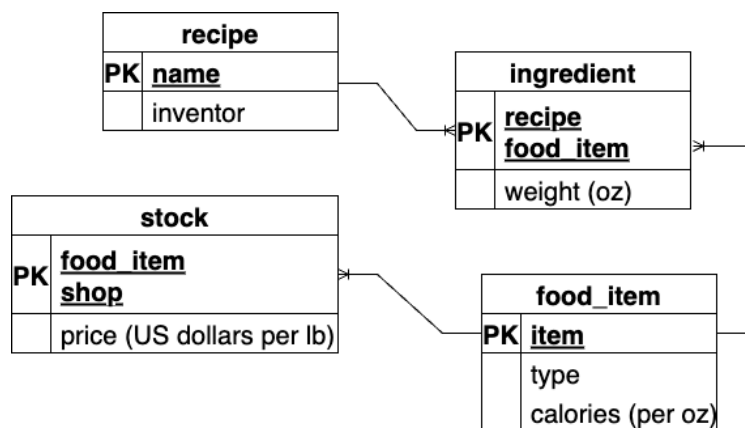
- **Your .Rmd file must be knittable. If your .Rmd file does not knit in graders' computer, you will receive at most half credit for the assignment.**
The two most common reason files fail to knit are because of workspace/directory structure issues and because of missing include files. To remedy the first, ensure all of the file paths in your document are relative paths pointing at the current working directory. To remedy the second, simply make sure you upload any and all files you source or include in your .Rmd file.
- Your coding should adhere to the tidyverse style guide: <https://style.tidyverse.org/>.

NOTE: *Everything* you need to do this assignment is here, in your class notes, or was covered in discussion or lecture.

- **DO NOT** look for solutions online.
- **DO NOT** collaborate with anyone inside (or outside) of this class.
- Work **INDEPENDENTLY** on this assignment.
- **EVERYTHING** you submit **MUST** be 100% your, original, work product. Any student suspected of plagiarizing, in whole or in part, any portion of this assignment, will be **immediately** referred to the Dean of Student's office without warning.

1: Cook's relational data

- Consider the following entity-relationship diagram and corresponding CSV files¹.
- Please use only the functions from the **dplyr** package to answer the questions below.
- Don't use any control statements. No points will be awarded for solutions that use control statements.
- Use only one linear sequence of pipe operations to answer each question.
- The efficiency and accuracy of your code account for 40% of this assignment.



- Write R code to return the types of food and the total calories for each type to make a turkey burger from the recipe.
- Write R code to return all recipes containing both bread and tomato products with their weights on these two ingredients.
- Write R code to return the total calories of the beef and turkey burgers.
- Write R code to return all vegetable items sold at either 'W-Mart' or 'Food Warehouse' and display which shop offers the lowest price for each item.
- Write R code to return the recipes with the total calories that contain "Wheat product".

¹you can find them on [bruinlearn](#)

2: Regular Expression Exercises

- The exercises focus on using regular expressions and the stringer functions.
 - Don't use any control statements.
 - Please find and download the corresponding files on the course website to complete the tasks below.
 - For each pattern, store the pattern as a variable named in the way **pat_x_y** where the x is the number of the exercise and the y is the letter of the sub-exercise.
 - You should save these patterns in your **.R** file.
 - You may demonstrate your patterns with any appropriate stringer functions.
 - The accuracy of your patterns account for 60% of this assignment.
1. Find names.txt and write regex patterns to answer following questions.
 - a. Find all usernames that contain at least one numeric character.
 - b. Find all usernames that are exactly four characters long and consist only of alphabetic characters.
 - c. Find all usernames following the conventional name format, where the given name precedes the family name, with optional additional names in-between. Each name segment is separated by a single white space and should begin with an uppercase letter followed by one or more lowercase letters. Note that intermediate names should also start with an uppercase letter followed by zero or more lowercase letters.
 2. Find cards.txt and write the regex patterns to identify all the valid card numbers with the formatting rules below².
 - A Master card number begins with a 5 and it is exactly 16 digits long.
 - A Visa card number begins with a 4 and it is between 13 and 16 digits long.
 - a. Write a regex pattern to match valid Master card number and print all the valid numbers, grouped into sets of 4 separated by a single space.
 - b. Write a regex pattern to match valid Visa card number and print all the valid numbers, grouped into sets of 4 separated by a single space.

Note: You may use more than one stringer function for both questions above.
 3. Find the passwords.txt and write regex patterns to answer the following questions.
 - a. Write a regex pattern to identify the passwords that satisfies the requirements below.
 - Minimum 8 characters

²Typically, we will need the formulas to validate real credit card numbers, but you don't have to worry about it for this exercise.

- Must contain at least one letter
 - Must contain at least one digit
- b. Write a regex pattern to identify the passwords that satisfies the requirements below.
- Minimum 8 characters
 - Must contain at least one uppercase character
 - Must contain at least one lowercase character
 - Must contain at least one digit
 - Must contain at least one punctuation character
4. Please load wordlists.RData to R and write regular expression patterns which will match all of the values in x and none of the values in y.

a. **Ranges**

	x	y
1	abac	beam
2	accede	buoy
3	adead	canjac
4	babe	chymia
5	bead	corah
6	bebed	cupula
7	bedad	griece
8	bedded	hafter
9	bedead	idic
10	bedeaf	lucy
11	caba	martyr
12	caffa	matron
13	dace	messrs
14	dade	mucose
15	daff	relose
16	dead	sonly
17	deed	tegua
18	deface	threap
19	faded	towned
20	faff	widish
21	feed	yite

b. Backrefs

	x	y
1	allochirally	anticker
2	anticovenanting	corundum
3	barbary	crabcatcher
4	calelectrical	damnably
5	entablement	foxtailed
6	ethanethiol	galvanotactic
7	froufrou	gummage
8	furfuryl	gurniad
9	galagala	hypergoddess
10	heavyheaded	kashga
11	linguatuline	nonimitative
12	mathematic	parsonage
13	monoammonium	pouchlike
14	perpera	presumptuously
15	photophonic	pylar
16	purpuraceous	rachioparalysis
17	salpingonasal	scherzando
18	testes	swayed
19	trisectrix	unbridledness
20	undergrounder	unupbraidingly
21	untaunted	wellside

c. Prime

	x	y
1	xx	xxxx
2	xxx	xxxxxx
3	xxxxx	xxxxxxxx
4	xxxxxxx	xxxxxxxxxx
5	xxxxxxxxx	xxxxxxxxxx
6	xxxxxxxxxxx	xxxxxxxxxx
7	xxxxxxxxxxxxx	xxxxxxxxxx
8	xxxxxxxxxxxxxxx	xxxxxxxxxx
9	xxxxxxxxxxxxxxxxx	xxxxxxxxxx
10	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
11	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
12	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
13	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
14	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
15	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
16	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
17	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
18	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
19	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx
20	xxxxxxxxxxxxxxxxxxx	xxxxxxxxxx